CS482 Software Project:
5-Draw Poker

Cyrus McCollim, Heloisa Albuquerque, Charles Humphreys, Sean Triplett

2026-10-25

1 Client Information

- Client name: Dr. Megan Olsen
- Client title: Professor
- Client email address: mmolsen@loyola.edu
- Client employer: Loyola University Maryland
- How you know the client: Professor

2 Project Description

2.1 Overview

[Add a few paragraphs describing your project succinctly. What problem are you trying to solve, what is the purpose of your project? Why does your client want this project?]

2.2 Key Features

[At this point you should have a basic understanding of your client's needs. List out the key features of the software system the client wants you to build.]

2.3 Why this Project is Interesting

[Why did you decide this project was interesting enough to you to be a capstone project? What about this project is enticing? Why should anyone care?]

2.4 Areas of CS required

[What subfields of computer science seem most likely to be relevant to your project? A capstone must involve multiple.]

2.5 Potential Concerns and Questions

[Is there any aspect of this project that makes you unsure if it will work, either due to your own interests/background, or that you aren't sure if it fits the requirements? Are there questions you have about this project that you want instructor feedback about?]

2.6 Summary of Efforts to Find a Project

(Not necessary for 482) [Briefly list out when/how you've discussed with this client, and if you've discussed with other clients who either didn't work out or didn't respond. If you considered a different project and it didn't work out, why didn't it work out?]

[Most CS495 projects end here. The sections below are for CS482 and CS496 software projects].

2.7 Comparison to Draft

[For CS496 only, focus on highlighting the major differences between the draft proposal in CS495 and this one here. If there are no major differences, you can remove this subsection.]

3 Requirements

3.1 Non-Functional Requirements

Table 1 presents the NFRs for this software project.

ID	NFR Title	Category	Description
1	Aesthetics	Design	The color scheme is red and assets used for backgrounds, buttons, cards, and other elements are for a mature audience.
3	Time Outs	Gameplay	System automatically kicks users taking too long on their turn and deletes inactive rooms.
4	Money	Reliability	The system accurately processes and updates balances and payouts without loss (including during multiplier events).
6	Rate Limits	Security	System has rate limits for sensitive actions like guest sign in, lobby creation, clicking ads, sending messages.
7	Availability	Reliability	The site is hosted to be highly available for play at most times.
8	Mobile	Gameplay	The site and game are functional on mobile devices.
10	Sound	Design	System plays music and sound effects for game interactions and events.

Table 1: Non-Functional requirements

3.2 Functional Requirements (User Stories)

Table 2 presents the functional requirements for this software project.

ID	Story Title	Points	Description
S1	Game Initialization	2	As an Player, I want us to be dealt 5 cards after putting our antes into the pot, so that the first round can begin.
S2	Game Betting	2	As a/an Player, I want to be able to Check, Call, Bet/Raise, or Fold, so that I can make a decision during a betting round.
S3	Game Card Draw	2	As a/an Player, I want to select between 0 and 5 cards to discard, so that I can replace them with new cards from the deck during the draw round.
S4	Game Ending	2	As a/an Player, I want all player hands to be compared after the final betting round, so that the pot is awarded to the winner.
S5	Bot Play	5	As a/an Player, I want to add bots to the room, so that I can play 5 Card Draw Poker when not enough people are online.

S6	Sign Up	2	As a/an Player, I want to create an account, so that I can save my progress.
S7	Sign In / Sign Out	1	As a/an Player, I want to sign in / sign out to my account, so that I can pick up where I left off.
S8	User Settings	3	As a/an Player, I want to update my own profile, so that I can customize how I appear to others.
S9	Guest Player	1	As a/an Player, I want to login as a guest, so that I can play without an account.
S10	View Balance	1	As a/an Player, I want to view my balance, so that I can know how much I have won.
S11	View My History	2	As a/an Player, I want to see my game history, so that I can see my performance over time.
S12	Room Creation	2	As a/an Player, I want to create a room, so that I can play with others.
S13	Room Settings	3	As a/an Admin, I want to adjust room settings, so that I can change the max players, set minimum bets, and configure raise per X rounds.
S14	Room Joining	2	As a/an Player, I want to join a room, so that I can play with others.
S15	Chat Feature	5	As a/an Player, I want to be able to message in a chat room, so that I can communicate with other players during a game.
S16	Contact Admin	1	As a/an Player, I want to message the admin, so that I can report any problems.
S17	Advertisements	1	As a/an Player, I want to click on ads, so that I can get more in game money.
S18	Leaderboard	1	As a/an Player, I want to see a leaderboard of the top balance players, so that I can see where I stand.
S19	Daily Rewards	1	As a/an Player, I want to get daily rewards, so that I have a reason to log in every day.
S20	Rank Badge	1	As a/an Player, I want to receive a badge when I reach the top of the leaderboard, so that my profile is distinguished as a top player.
S21	Cosmetic Shop	5	As a/an Player, I want to buy cosmetic items, so that I have something to spend my in game money on.
S22	Promotional Event	1	As a/an Admin, I want to trigger special events with chip multipliers, so that I can boost player engagement.
S23	Remove Users	2	As a/an Admin, I want to delete a user, so that I can remove malicious players.
S24	Reset Password	1	As a/an Admin, I want to reset a user's password, so that a player is not locked out if they forget it.
S25	View All Profiles	2	As a/an Admin, I want to see all player game histories, so that I can inspect games for problems.
S26	Modify Chips	1	As a/an Admin, I want to modify a player's balance, so that I can reimburse them for problems.
S27	Adjust Volume	1	As a/an Player, I want to change volume levels for music and sound effects, so that I can choose what I hear.
S28	Friends List	3	As a/an Player, I want to have a friends list that I can manage, so that I can play with my friends again, add them, or remove them.
S29	Advertisements	1	As a/an Admin, I want to create ads, so that players can have ads to click on to get in game money.
	Total:	57	

Table 2: Functional Requirements (User Stories)

3.3 AI Analysis on Functional Requirements

[After you receive your revised User Stories in CS482, or after your first delivery of Stories on CS496, you should use AI as a senior software engineer to help you find any missing or hidden requirements.] [You must provide which LLM you use, which prompt you used, and most importantly *reflect* on the results the LLM gave back. Just because the LLM suggested something, it does not mean it was a valid suggestion and that you should follow blindly. Use your knowledge and reason on which ones are reasonable to add to your requirements and which ones are not.]

4 System Design

4.1 Architecture

We will use the MVC architecture to structure this application. Two servers will be hosted for the frontend and backend respectively. The frontend will serve the HTML/CSS/JS which includes graphics, menus, buttons, animations, and interface logic. The backend will be the source of truth by validating requests, updating game states, and interacting with the database. This should help with persistent data and real-time multiplayer interactions. The backend separates the game engine from the network and database layers to ensure that the card mechanics for 5-card poker can be unit tested. The main modular components for the application include Auth, Player, Profile, Friends, Admin, Lobby, Room, Chat, Game State, Hand Evaluator, Bot, Wallet, Shop, History, Leaderboard, Ads, and Events.

4.2 Diagrams

[CS482, on sprints/iterations 2-3, you need to create and update a diagram (check Iteration 2-3 assignment for which type of diagram). On CS496, since before sprint/iteration 1, you should have a class diagram and keep it up-to-date. In CS496, if your class diagram changes at each sprint, then create a Class Diagram subsection for the sprints, and show the changes; while keeping the one here the most up-to-date version.]

4.3 Technology

For our programming language, we chose **TypeScript** for both the front and back end because of its strict type enforcement makes unit testing and error checking easier. For our front end, we will use **React** bootstrapped with **Next.js** (written in TypeScript) for our UI, which has built-in file-based routing. For our backend, we will use **Node.js** with **Express** and a **REST API** for accounts, friends, tables, and adverts compiled from typescript. For real-time communication, we will use **Socket.IO** to manage live gameplay updates. Each poker table is a room, and Socket.IO's built-in reconnection handles dropped players. It also comes shipped with its own TypeScript types. For our database, we chose **Supabase**, a Postgres-based database that provides relational storage for profiles, friendships, tables, hand history, and adverts. For authentication, we will use **Supabase Auth** for account creation, login, and password handling (server verifies user tokens). For hand evaluation, we have chosen to use an open source poker solving library. We've tested its ability to rank hands to determine winners (with ties at showdown), and we verified its ability to handle rank, tie, wheel straight, and kicker cases. For our configuration, we will use **dotenv** to keep API keys and secrets out of the source code. For our version control, we will use **Git** and **GitHub**, as it will allow us to create feature branches and pull request reviews. For our UI sketches and mockups we decided to use **Figma** as it provides a high flexibility. We will use **Jest** to run our tests for both the front and the back end, with the **React Testing Library** for components and **Supertest** for REST routes. We'll test the Socket.IO behavior by connecting test clients to a test server instance, and we'll use Jest's built-in coverage report.

4.4 Coding Standards

Each group member will add comments to clarify their code when appropriate. The `any` type will be avoided in TypeScript. Code files will be named with singular capitalized words, and database tables and columns will use singular snake case names. All group members will meet at least once per sprint. Each sprint, we

will use the following merge flow: feature branch → develop branch (requires 1 PR) → main branch (requires 2 PRs). These branch protection rules encourage code reviews and help avoid messy repository states. No restrictions apply while a member is working on their own feature branch. For each merge to develop, 100% of unit tests must pass and there must be at least 80% code coverage. Each test must have a clearly defined purpose, and coverage will be visible through CI/CD workflows set up with GitHub Actions. When a bug is discovered later on, tests should be created for it to prevent regression in the future. Every pull request must be approved by a teammate other than its author, and no one may push directly to the develop or main branches. Branches will be named by the issue number and feature (for example, `feature/13-login-page`), and commit messages will also contain the issue number, be short, and describe the change.

4.5 Data

The database is built around players and the games they play. The user table is the center, and each user has a username, a chip balance, and a few things like a daily reward streak. Players sit at a table, which is a game room with a host, a buy-in, a max number of players, and betting rules. The **table_players** table connects users to tables and tracks which seat they have and how many chips they brought. Each round of play at a table is saved in the match table, which stores the pot and who won. **match_players** lists everyone in that match and how much they won or lost, and **match_log** records every move they made, like bet, call, raise, fold, check, or draw, so a game can be replayed later. Around this core are the extras: friendship links two users together, **shop_item** and **user_inventory** let players buy and equip things with chips, advert and **ad_claim** let players earn chips by watching ads, and event handles special bonuses. **chat_message** stores table chat, **support_ticket** stores help requests, and **balance_log** keeps a history of every chip change so nothing gets lost.

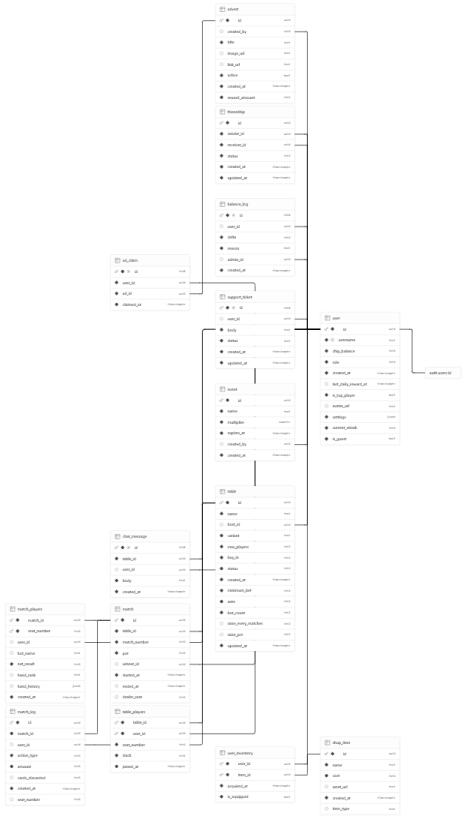
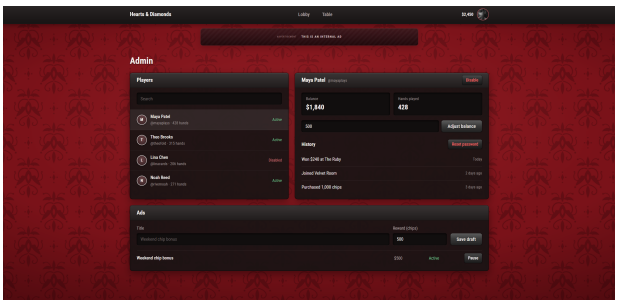


Figure 1: ER diagram of Database Schema

Table	Essential Columns
user	id, username, balance, role, current_streak
friendship	id, sender_id, receiver_id, status
table	id, name, host_id, max_players, buy_in, minimum_bet, status
table_players	table_id, user_id, seat_number, stack
match	id, table_id, match_number, pot, winner_id
match_players	match_id, seat_number, user_id, net_result, hand_rank
match_log	id, match_id, user_id, action_type, amount
shop_item	id, name, cost, item_type
user_inventory	user_id, item_id, is_equipped
advert	id, title, link_url, active, reward_amount
ad_claim	id, user_id, ad_id, claimed_at
event	id, name, multiplier, expires_at
chat_message	id, table_id, user_id, body
support_ticket	id, user_id, body, status
balance_log	id, user_id, delta, reason

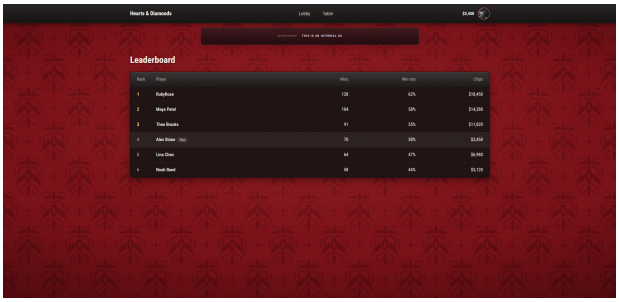
4.6 UI Mocks

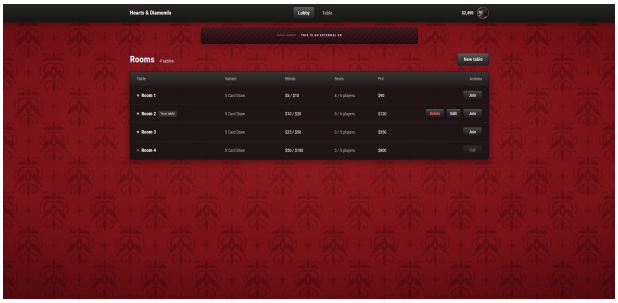
All pages will have a mature red theme. On the gameplay page, a red oval in the middle will act as the poker table. For players, the UI will include a login and sign up page, a page for joining and creating lobbies, the game room, a user profile, and, as a stretch goal, a shop page. For admins, the UI will include a login page, a settings page, a page for spectating rooms, an admin profile, a shop page, and a page for searching through players' profiles.

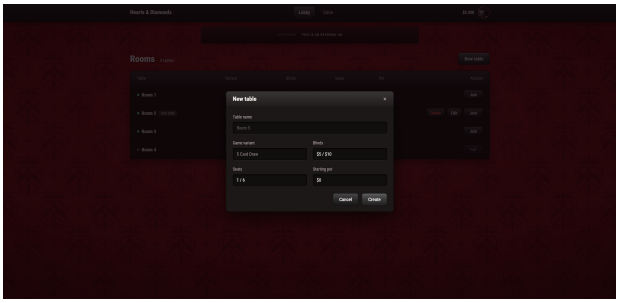


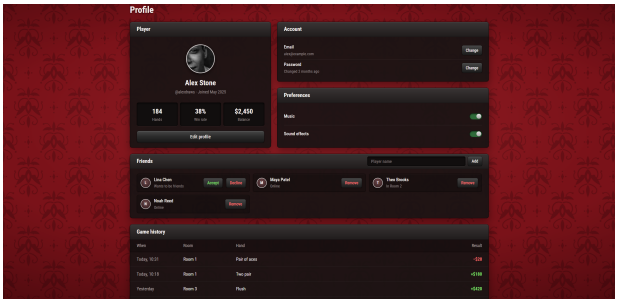




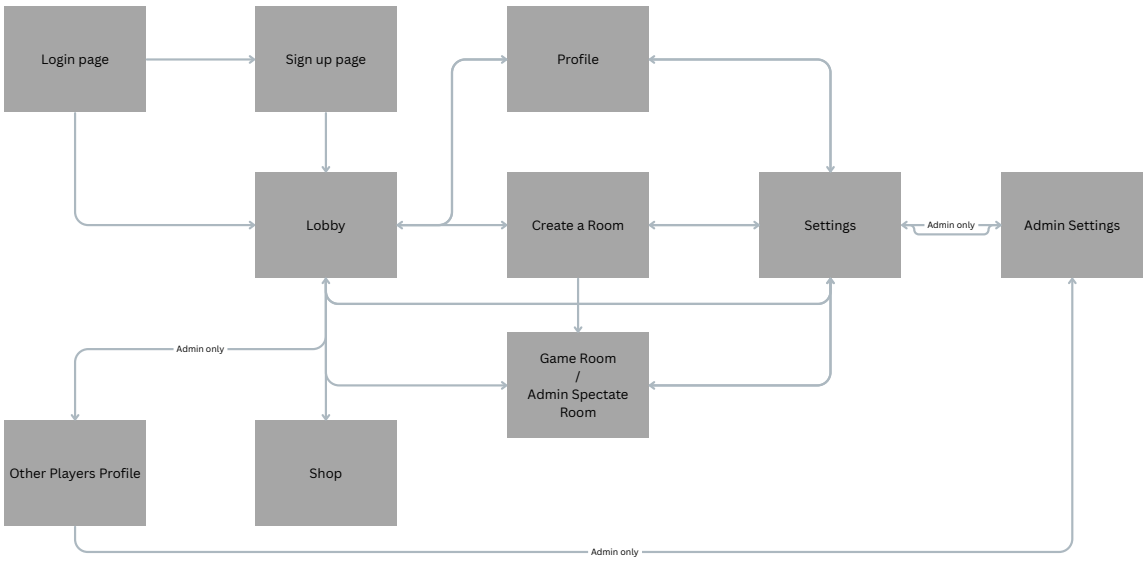












5 Iterations

5.1 Iteration Planning

[In CS496, you plan all iterations beforehand. In CS482, you update the planning here at each iteration.]

Table 3 shows the iteration planning.

It.	Dates	Stories	Points Planned	Done
1	01/01 - 02/01	S1 Story Example, S2 Story Example 2	07	05
2	02/01 - 03/01	S3 Story Title, S4 Story Title, S5 Story Title, S6 Story Title	17	15
3	03/01 - 04/01	S7 Story Title, S8 Story Title, S9 Story Title, S10 Story Title, S11 Story Title	21	20
4	04/01 - 05/01	S12 Story Title, S13 Story Title, S14 Story Title, S15 Story Title	19	19
5	05/01 - 06/01	S16 Story Title, S17 Story Title	06	06
		Total:	70	65

Table 3: Iteration Planning for Incremental Deliveries

5.2 Iteration/Sprint 1

5.2.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.2.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.2.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

Figure 2 shows the test coverage

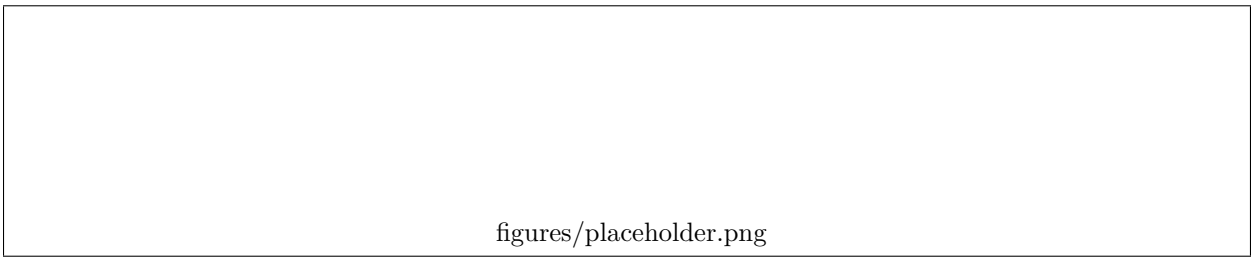


Figure 2: Iteration 1 test coverage report

5.2.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.3 Iteration/Sprint 2

5.3.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.3.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.3.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.3.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.4 Iteration/Sprint 3

5.4.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.4.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.4.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.4.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.5 Iteration/Sprint 4

[CS496 has 5 sprints. CS482 only has only 3 sprints (remove Iterations 4 and 5 from this doc if you are writing a doc for 482]

5.5.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.5.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.5.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.5.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.6 Iteration/Sprint 5

5.6.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.6.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.6.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.6.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

6 Final Remarks

6.1 Overall Progress

[Have you completed everything? If so, present evidence on how you brought value to your client, and the overall client satisfaction. Otherwise, estimate how much progress you done and how long it would take to finish this project. Be concrete about your progress, you know how many story points your software is, how many points you completed (this shows your progress). You also how many points your team delivers at each iteration, therefore you can estimate how many more iterations it would take to finish the leftover points (show the math).]

6.2 Project Reflection

[Your personal reflection on the project. What lessons did you learned. What would you have done differently? How can you do better work in future projects? You may write this as a team or per person (or both — if all your iterations were team reflections, then it would be better to write individual reflections here)]

Appendix

[Appendix section if needed]